Data Engineer

Globant Challenge

Table of Contents

Data Engineering Challenge

Context

You are a Data Engineer at Globant working on a data migration project to a new SQL-based system.

The goal of this challenge is to evaluate your ability to work with:

- Data ingestion
- Data validation
- SQL modeling and querying
- API development
- Basic data engineering practices

Challenge #1: Data Ingestion & API

You must build a proof of concept (PoC) to satisfy the following requirements:

1. Historical Data Migration

Load historical data from CSV files into a SQL database.

You are free to choose:

- The database (must be SQL-based)
- The location of the CSV files
- The ingestion strategy

2. REST API for Data Ingestion

Build a REST API to receive new data for all tables.

Requirements:

- Each request must validate the data according to the data dictionary
- Support batch ingestion (from 1 up to 1000 rows per request)
- The API must support ingestion for all tables:
  - hired_employees
  - jobs
  - departments
- You may implement:
  - Separate endpoints per table, or
  - A generic endpoint (justify your decision)

3. Data Validation Rules

Important: The dataset contains invalid records intentionally.

You must enforce the following rules:

- All fields are required:
  - id
  - name
  - datetime
  - department_id
  - job_id
- datetime must be in ISO format (e.g. `2021-07-27T16:02:08Z`)
- department_id must exist in departments table
- job_id must exist in jobs table
- Records that do not comply:
  - MUST NOT be inserted

- MUST be logged (you define the logging mechanism)

4. Backup Feature

Implement a feature to:

- Export the full content of each table
- Save it in AVRO format on the filesystem

5. Restore Feature

Implement a feature to:

- Restore a table from an AVRO backup file

Clarifications

- CSV files are comma-separated
- You may use any architecture/approach (scripts, APIs, jobs, stored procedures, etc.)
- Features can be implemented in any reasonable way

Not mandatory (but evaluated)

- README.md with clear instructions
- API security considerations
- Git workflow (commits, branches)
- Docker support
- Use of cloud services

Data Model

hired_employees.csv

field	type	description
id	INTEGER	Employee ID
name	STRING	Employee full name
datetime	STRING	Hire datetime (ISO 8601)
department_id	INTEGER	Department ID
job_id	INTEGER	Job ID

departments.csv

field	type	description
id	INTEGER	Department ID
department	STRING	Department name

jobs.csv

field	type	description
id	INTEGER	Job ID
job	STRING	Job name

Challenge #2: Data Analysis (SQL + API)

You must expose an endpoint for each of the following queries.

Important:

- Only consider records from year **2021**
- Only valid records should be used (after validation from Challenge #1)

1. Employees hired per job and department by quarter

Return:

- Number of employees hired for each job and department in 2021
- Aggregated by quarter (Q1â€"Q4)
- Ordered alphabetically by department and job

Expected output structure:

department | job | Q1 | Q2 | Q3 | Q4

Example (illustrative structure only)

Accounting | Software Engineer I | 2 | 5 | 3 | 4
Engineering | Data Coordinator | 1 | 2 | 2 | 1
Human Resources | Recruiter | 0 | 3 | 1 | 2

Note:

- Values will depend on your cleaned dataset
- Do not hardcode results

2. Departments above average hiring

Return:

- Departments that hired more employees than the average across all departments in 2021

Output:

id | department | hired

Example (structure only, values depend on dataset):

5 | Engineering | 120
9 | Marketing | 95
2 | Sales | 90

Ordered by:

- hired DESC

(Optional) Visualization

You may include charts or dashboards using any BI tool.

Evaluation Criteria

Area	Description
Data Validation	Correct handling of invalid records
SQL	Correctness and efficiency of queries
API Design	Clarity, usability, structure
Code Quality	Readability, structure, best practices
Performance	Reasonable efficiency
Documentation	Clarity of README and instructions

What We Are Evaluating

This challenge focuses on:

- Basic Python skills (data processing, APIs)
- SQL proficiency (joins, aggregations, filtering)
- Data quality handling
- Ability to reason about real-world messy data

Final Notes

- The dataset intentionally includes invalid and inconsistent data
- Your solution should handle these scenarios gracefully
- Prioritize correctness and clarity over over-engineering